Algoritmi ricorsivi (1/2)

Sapienza Università di Roma

Indice

La somma degli elementi di un array

Merge Sort

Dai capitoli precedenti

Nei capitoli «Ricorsione», «Equazioni di ricorrenza» e «Il Master theorem» abbiamo imparato:

- come viene eseguito un algoritmo ricorsivo
- come se ne descrive il numero di passi con un'**equazione di ricorrenza**
- come si risolve tale equazione

In questo capitolo mettiamo al lavoro questi strumenti su quattro problemi classici sugli array.

Quattro problemi classici sugli array

- la **somma** degli elementi di un array, il problema più semplice possibile, su cui confrontiamo un algoritmo iterativo e due algoritmi ricorsivi
- l'**ordinamento**, con il **Merge Sort**, il primo algoritmo del corso che batte gli algoritmi di ordinamento $\Theta(n^2)$
- la **ricerca** in un array ordinato, con la **ricerca binaria ricorsiva**
- il problema del **sotto-array di somma massima**, in cui un algoritmo divide et impera migliora la forza bruta, e un algoritmo iterativo ingegnoso migliora entrambi

In questa lezione: la somma di un array e il Merge Sort.

Lo schema dell'analisi

Tutti gli algoritmi sono studiati con lo stesso schema, che è lo schema dell'analisi di ogni algoritmo ricorsivo. Per ogni algoritmo ricorsivo:

1. scriviamo lo pseudocodice e lo eseguiamo su un piccolo esempio, disegnando l'**albero delle chiamate**;
2. dimostriamo che è **corretto**, per induzione (forte) sulla dimensione del sottoproblema;
3. descriviamo il suo numero di passi con un'**equazione di ricorrenza** e la risolviamo, con l'albero di ricorsione e con il Master theorem;
4. esprimiamo il **tempo di esecuzione** nel caso peggiore in funzione della dimensione $|I|$ dell'input;
5. misuriamo la **memoria aggiuntiva**, che per un algoritmo ricorsivo comprende la pila delle chiamate;
6. diamo un'implementazione (array indicizzati da 0, mentre nello pseudocodice sono indicizzati da 1): i notebook Colab in Python, C++ e Java.

Divide et impera

- Tre dei quattro algoritmi seguono il paradigma **divide et impera**:
  - il problema su un sotto-array $a[l..r]$ viene suddiviso in sottoproblemi sulle due metà $a[l..m]$ e $a[m+1..r]$ (*divide*)
  - i sottoproblemi vengono risolti ricorsivamente (*impera*)
  - le loro soluzioni vengono messe insieme (*combina*)

- La ricerca binaria ricorsiva, invece, effettua una **sola** chiamata ricorsiva su una metà

La somma degli elementi di un array

Introduzione: La somma degli elementi di un array

- Il problema più semplice sugli array: lo usiamo per confrontare un algoritmo **iterativo** e un algoritmo **ricorsivo** che risolvono lo stesso problema
- Su questo problema percorriamo l'intero schema dell'analisi: pseudocodice, albero delle chiamate, correttezza, equazione di ricorrenza, albero di ricorsione, Master theorem, tempo di esecuzione
- Scopriamo che la ricorsione non rende l'algoritmo più veloce, e che la **pila delle chiamate** costa memoria

Il problema

Dato un array $a[1..n]$ di $n \geq 1$ numeri interi, consideriamo il problema di calcolare la somma dei suoi elementi:

$$\sum_{i=1}^{n} a[i] = a[1] + a[2] + \cdots + a[n].$$

- È il problema ideale per confrontare un algoritmo **iterativo** e uno **ricorsivo**
- Scriviamo l'algoritmo ricorsivo, dimostriamo che è corretto, descriviamo il suo numero di passi con un'equazione di ricorrenza e risolviamo l'equazione, con gli strumenti dei capitoli «Ricorsione», «Equazioni di ricorrenza» e «Il Master theorem»

Esempio

Consideriamo il seguente array di $n = 7$ numeri interi:

$a[i]$	$i = 1$	$i = 2$	$i = 3$	$i = 4$	$i = 5$	$i = 6$	$i = 7$
	9	8	12	3	5	14	6

La somma dei suoi elementi è

$$9 + 8 + 12 + 3 + 5 + 14 + 6 = 57.$$

Algoritmo iterativo

Partiamo dall'ovvio algoritmo iterativo `SumIterative`,
che scorre l'array da sinistra a destra
e accumula le somme parziali nella variabile s.

Algorithm: SumIterative

Input : Un array $a[1..n]$ di n numeri interi
Output : La somma $a[1] + a[2] + \cdots + a[n]$

```
1 function SumIterative(a, n)
2     s := 0
3     i := 1
4     while i ≤ n do
5         s ← s + a[i]
6         i ← i + 1
7     end
8     return s
9 end
```

Esegui in Colab: Python · C++ · Java

Numero di passi di `SumIterative`

- Sia $S(n)$ il numero di passi eseguiti da `SumIterative` su un array di n elementi (non c'è differenza tra caso migliore e caso peggiore: l'algoritmo esegue sempre le stesse operazioni)

riga	numero di esecuzioni
2	1
3	1
4	$n+1$
5	n
6	n
8	1

Passi e memoria di `SumIterative`

- Quindi,

$$S(n) = 1 + 1 + (n+1) + n + n + 1 = 3n + 4 = \Theta(n),$$

poiché

$$\lim_{n\to\infty} \frac{3n+4}{n} = \lim_{n\to\infty} \left(3 + \frac{4}{n}\right) = 3 \in \mathbb{R}_{>0}.$$

- Oltre all'array in ingresso, l'algoritmo usa solo le due variabili s e i: la **memoria aggiuntiva** è $\Theta(1)$

Algoritmo ricorsivo (divide et impera)

- La stessa somma si può calcolare ricorsivamente. La somma degli elementi di un sotto-array $a[l..r]$ si ottiene con il metodo **divide et impera**:
  1. **Divide**: si suddivide $a[l..r]$ nelle due metà $a[l..m]$ e $a[m+1..r]$, dove $m = \lfloor (l+r)/2 \rfloor$;
  2. **Impera**: si calcolano ricorsivamente la somma s_1 di $a[l..m]$ e la somma s_2 di $a[m+1..r]$;
  3. **Combina**: si restituisce $s_1 + s_2$.
- Il **caso base** è un sotto-array con un solo elemento ($l = r$): la sua somma è l'elemento $a[l]$ stesso
- Nella prossima slide, lo pseudocodice dell'algoritmo ricorsivo `RecursiveSum`. Per sommare l'intero array chiamiamo `RecursiveSum`$(a, 1, n)$

Algorithm: RecursiveSum

Input : Un array a e due indici $l \leq r$
Output : La somma $a[l] + a[l+1] + \cdots + a[r]$

```
1 function RecursiveSum(a, l, r)
2     if l = r then
3         return a[l]                                  // Caso base
4     end
5     m := ⌊(l + r)/2⌋                                 // Divide
6     s1 := RecursiveSum(a, l, m)                      // Impera (metà
       sinistra)
7     s2 := RecursiveSum(a, m + 1, r)                  // Impera (metà
       destra)
8     return s1 + s2                                   // Combina
9 end
```

Esegui in Colab: Python · C++ · Java

La ricorsione raggiunge sempre il caso base

- Sia $k = r - l + 1$ il numero di elementi di $a[l..r]$
- Se $k \geq 2$, le due metà hanno

$$m - l + 1 = \left\lceil \frac{k}{2} \right\rceil \qquad \text{e} \qquad r - m = \left\lfloor \frac{k}{2} \right\rfloor$$

  elementi, rispettivamente
- Entrambi i numeri sono almeno 1 e strettamente minori di k: ogni chiamata ricorsiva lavora su un sotto-array più piccolo, e la ricorsione raggiunge sempre il caso base

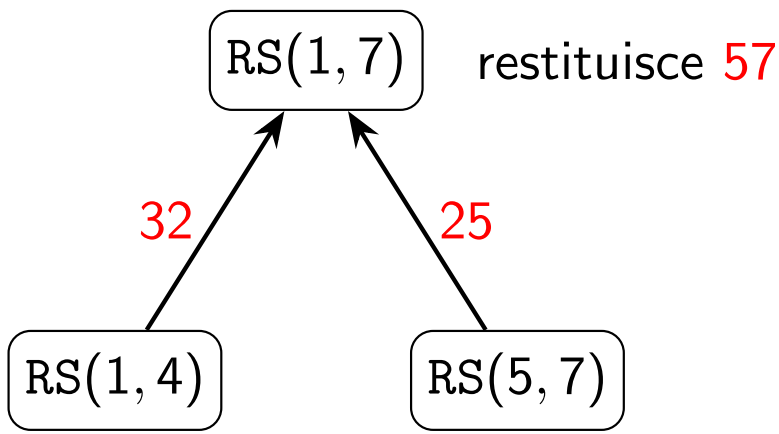
Albero delle chiamate

- L'esecuzione di un algoritmo ricorsivo è rappresentata dal suo **albero delle chiamate**:
  - ogni nodo è una chiamata
  - i figli di un nodo sono le chiamate che esso effettua
  - il valore restituito da una chiamata viene rimandato al padre

- Nel prossimo esempio disegniamo l'albero delle chiamate di `RecursiveSum` sull'array $a = [9, 8, 12, 3, 5, 14, 6]$, con $n = 7$

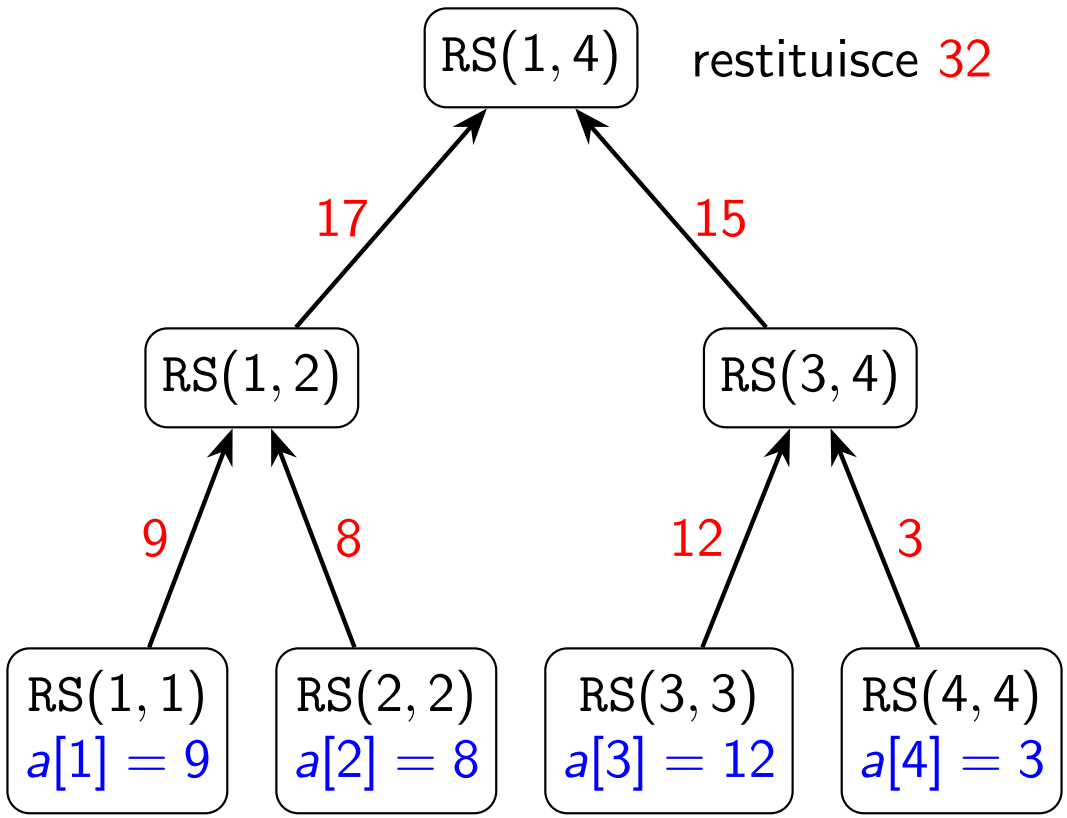
Esempio (1/4)

- L'albero delle chiamate di `RecursiveSum`$(a, 1, 7)$: ogni nodo RS(l, r) è una chiamata sul sotto-array $a[l..r]$
- In blu gli elementi dell'array restituiti dai casi base, in rosso i valori restituiti ai padri dalle chiamate (lungo le frecce)
- L'albero è troppo grande per una slide: disegniamo prima i suoi primi due livelli, poi il sottoalbero di RS$(1, 4)$ e il sottoalbero di RS$(5, 7)$



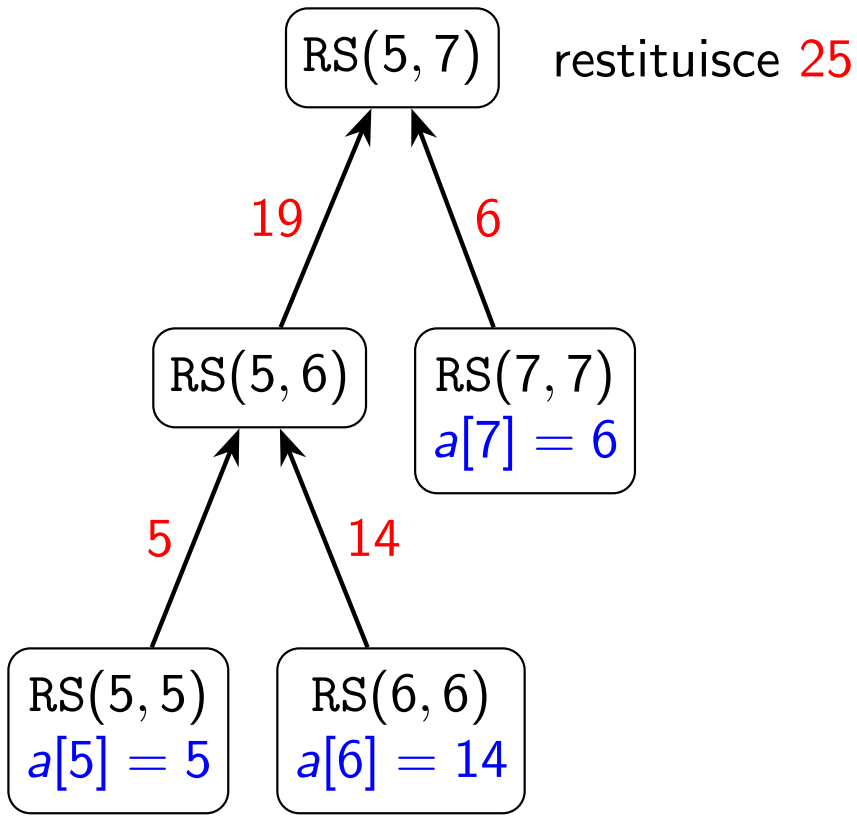
Esempio (2/4)

Il sottoalbero della chiamata $\texttt{RS}(1, 4)$, che restituisce 32 a $\texttt{RS}(1, 7)$:



Esempio (3/4)

Il sottoalbero della chiamata $\texttt{RS}(5, 7)$, che restituisce 25 a $\texttt{RS}(1, 7)$:



Esempio (4/4)

- La radice $\texttt{RS}(1, 7)$ calcola $m = \lfloor 8/2 \rfloor = 4$ e chiama $\texttt{RS}(1, 4)$ e $\texttt{RS}(5, 7)$
- $\texttt{RS}(1, 4)$ calcola $m = 2$ e restituisce $17 + 15 = 32$; $\texttt{RS}(5, 7)$ calcola $m = 6$ e restituisce $19 + 6 = 25$
- La radice restituisce $32 + 25 = 57$, che è la somma degli elementi dell'array
- Ci sono $13 = 2 \cdot 7 - 1$ chiamate: 7 foglie (casi base, una per ogni elemento) e 6 nodi interni, ciascuno dei quali esegue un'addizione, quindi in totale ci sono $6 = n - 1$ addizioni

Visita in profondità

- Le chiamate vengono eseguite in ordine di **visita in profondità**: una chiamata termina solo dopo che sono terminati entrambi i suoi figli

- Nell'esempio le chiamate terminano nell'ordine

$$(1,1),\ (2,2),\ (1,2),\ (3,3),\ (4,4),\ (3,4),\ (1,4),$$
$$(5,5),\ (6,6),\ (5,6),\ (7,7),\ (5,7),\ (1,7).$$

Correttezza

- La correttezza di un algoritmo ricorsivo si dimostra per **induzione forte** sulla dimensione del sottoproblema
- Si assume che le chiamate ricorsive, che lavorano su sotto-array più piccoli, siano corrette, e si mostra che la chiamata combina correttamente i loro risultati

Osservazione 1: correttezza di RecursiveSum

Per ogni coppia di indici $l \leq r$, *la chiamata* `RecursiveSum`(a, l, r) *termina e restituisce*

$$\sum_{i=l}^{r} a[i].$$

Dimostrazione (1/2)

Usiamo l'induzione forte sul numero $k = r - l + 1 \geq 1$ di elementi del sotto-array.

Caso base ($k = 1$). Allora $l = r$, la condizione della riga 2 è vera e l'algoritmo restituisce

$$a[l] = \sum_{i=l}^{l} a[i].$$

Passo induttivo ($k \geq 2$). Supponiamo che l'enunciato valga per tutti i sotto-array con meno di k elementi (*ipotesi induttiva*). Poiché $l < r$, l'algoritmo calcola $m = \lfloor (l + r)/2 \rfloor$, con $l \leq m < r$.

Dimostrazione (2/2)

I sotto-array $a[l..m]$ e $a[m+1..r]$ hanno $\lceil k/2 \rceil$ e $\lfloor k/2 \rfloor$ elementi, entrambi compresi tra 1 e $k-1$. Per l'ipotesi induttiva, le due chiamate ricorsive terminano e

$$s_1 = \sum_{i=l}^{m} a[i], \qquad s_2 = \sum_{i=m+1}^{r} a[i].$$

Quindi, l'algoritmo restituisce

$$s_1 + s_2 = \sum_{i=l}^{m} a[i] + \sum_{i=m+1}^{r} a[i] = \sum_{i=l}^{r} a[i].$$

□

- Con $l = 1$ e $r = n$, la chiamata `RecursiveSum`$(a, 1, n)$ restituisce la somma di tutti gli elementi dell'array

Numero di passi di `RecursiveSum`

- Sia $S(n)$ il numero di passi eseguiti da `RecursiveSum` su un sotto-array di $n = r - l + 1$ elementi. Come per l'algoritmo iterativo, dipende solo da n (e non dai valori degli elementi)
- Contiamo le righe di codice eseguite **da una singola chiamata**
- Una riga che contiene una chiamata ricorsiva conta 1 passo (la chiamata stessa) **più** tutti i passi eseguiti dalla funzione chiamata, che esprimiamo ancora con S
- Se $n = 1$, le righe 2 e 3 vengono eseguite 1 volta ciascuna, e quindi $S(1) = 2$

Numero di passi per $n \geq 2$

Se $n \geq 2$:

- La riga 2 viene eseguita 1 volta
- La riga 3 non viene eseguita
- La riga 5 viene eseguita 1 volta
- La riga 6 viene eseguita 1 volta, più i $S(\lceil n/2 \rceil)$ passi della chiamata su $a[l..m]$
- La riga 7 viene eseguita 1 volta, più i $S(\lfloor n/2 \rfloor)$ passi della chiamata su $a[m+1..r]$
- La riga 8 viene eseguita 1 volta

e quindi

$$S(n) = S(\lceil n/2 \rceil) + S(\lfloor n/2 \rfloor) + 5.$$

L'equazione di ricorrenza di `RecursiveSum`

Il numero di passi di `RecursiveSum` è descritto dall'equazione di ricorrenza

$$S(n) = \begin{cases} 2 & \text{se } n = 1, \\ S(\lceil n/2 \rceil) + S(\lfloor n/2 \rfloor) + c & \text{se } n \geq 2, \end{cases}$$

con $c = 5$.

Come di consueto, nell'analisi asintotica la scriviamo semplicemente come

$$S(n) = 2\,S(n/2) + c.$$

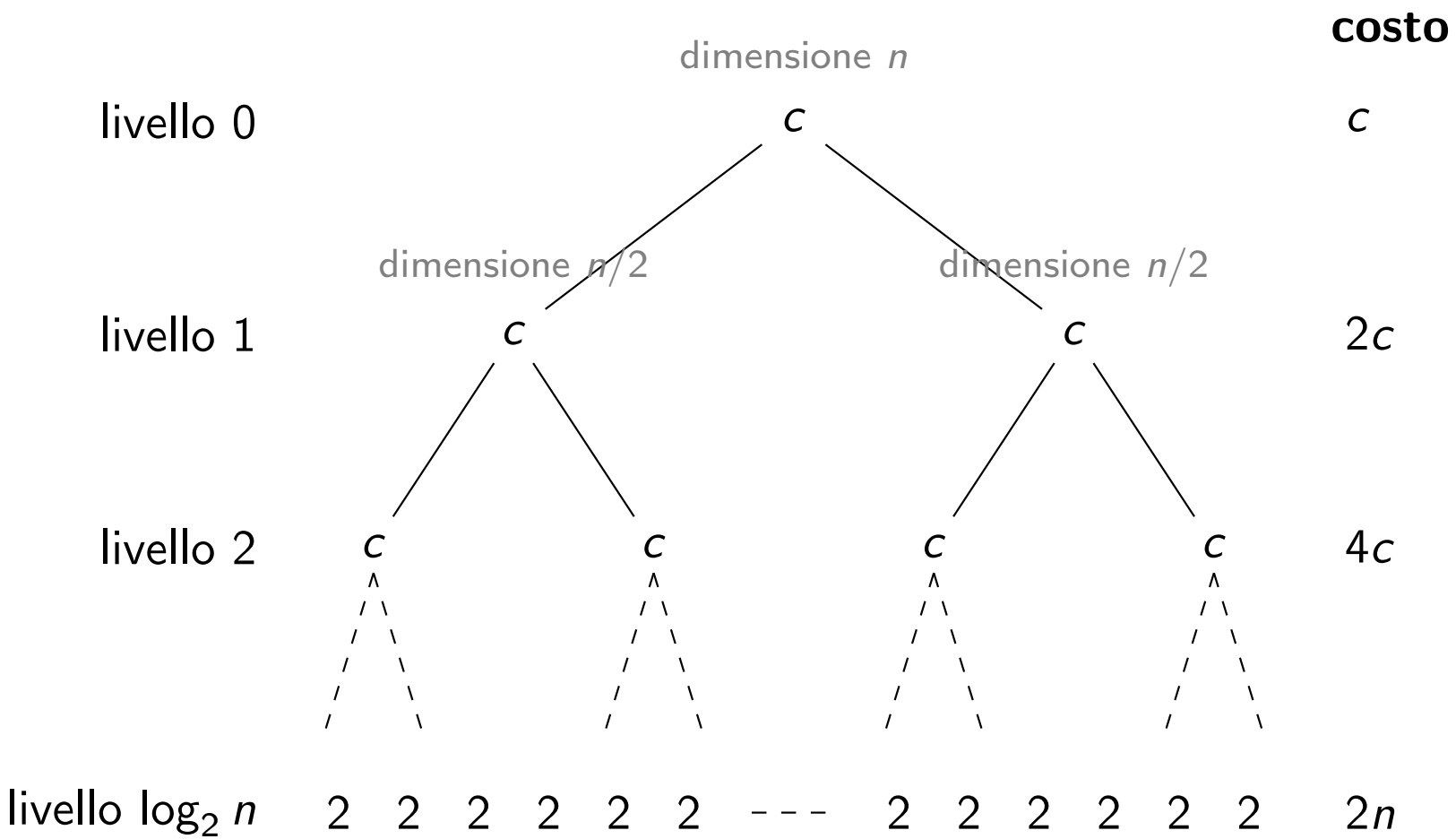
Soluzione con l'albero di ricorsione

- Nell'**albero di ricorsione** ogni nodo rappresenta una chiamata e contiene il numero di passi eseguiti da quella chiamata *esclusi* i passi delle sue chiamate ricorsive:
  - c per un nodo interno
  - 2 per una foglia

- $S(n)$ è la somma dei valori di tutti i nodi

- Per semplicità, supponiamo dapprima che $n = 2^h$ sia una potenza esatta di 2. Allora ogni sotto-array viene suddiviso in due metà di uguale dimensione, e l'albero è nella prossima slide

L'albero di ricorsione di $S(n) = 2\,S(n/2) + c$



Il conteggio dei nodi

- Il livello i contiene 2^i chiamate su sotto-array di $n/2^i$ elementi
- Le foglie sono al livello $\log_2 n$, dove i sotto-array hanno $n/2^{\log_2 n} = 1$ elemento: ci sono $2^{\log_2 n} = n$ foglie
- I livelli interni $0, 1, \ldots, \log_2 n - 1$ contengono

$$\sum_{i=0}^{\log_2 n - 1} 2^i = 2^{\log_2 n} - 1 = n - 1$$

  nodi (somma di una serie geometrica)

- Quindi il numero totale di passi è

$$S(n) = c\,(n-1) + 2n = 5(n-1) + 2n = 7n - 5.$$

Le foglie dominano

- Il costo dei livelli **raddoppia** da un livello al successivo: il totale è dominato dalle foglie, che sono n
- Questa è esattamente la situazione del **caso 1** del Master theorem
- Lo stesso conteggio vale per **ogni** n, non solo per le potenze di 2: il fatto chiave è che l'albero ha esattamente n foglie e $n-1$ nodi interni, come mostra la prossima osservazione

Osservazione 2: chiamate e passi di RecursiveSum

Per ogni $n \geq 1$, la chiamata `RecursiveSum`(a, l, r) su un sotto-array di $n = r - l + 1$ elementi genera esattamente $2n - 1$ chiamate (compresa la prima): n di esse sono casi base e $n - 1$ eseguono un'addizione. Inoltre, $S(n) = 7n - 5$.

Dimostrazione (1/2)

Sia $C(n)$ il numero di chiamate. Abbiamo $C(1) = 1$ e, per $n \geq 2$,

$$C(n) = 1 + C(\lceil n/2 \rceil) + C(\lfloor n/2 \rfloor).$$

Dimostriamo che $C(n) = 2n - 1$ per induzione forte su n.

Per $n = 1$: $C(1) = 1 = 2 \cdot 1 - 1$.

Per $n \geq 2$, per l'ipotesi induttiva applicata a $\lceil n/2 \rceil < n$ e $\lfloor n/2 \rfloor < n$,

$$\begin{aligned} C(n) &= 1 + \big(2\lceil n/2 \rceil - 1\big) + \big(2\lfloor n/2 \rfloor - 1\big) \\ &= 2\big(\lceil n/2 \rceil + \lfloor n/2 \rfloor\big) - 1 = 2n - 1, \end{aligned}$$

perché $\lceil n/2 \rceil + \lfloor n/2 \rfloor = n$ per ogni numero naturale n.

Dimostrazione (2/2)

Ogni caso base corrisponde esattamente a un elemento del sotto-array (i sotto-array delle foglie sono disgiunti e ricoprono $a[l..r]$), quindi ci sono n casi base e

$$2n - 1 - n = n - 1$$

chiamate che eseguono la riga 8.

Allo stesso modo, $S(1) = 2 = 7 \cdot 1 - 5$ e, per $n \geq 2$,

$$S(n) = \big(7\lceil n/2 \rceil - 5\big) + \big(7\lfloor n/2 \rfloor - 5\big) + 5 = 7n - 5.$$

□

$S(n)$ è lineare

- Di conseguenza,

$$S(n) = 7n - 5 = \Theta(n), \quad \text{poiché} \quad \lim_{n\to\infty} \frac{7n-5}{n} = 7 \in \mathbb{R}_{>0}.$$

- Per l'array dell'esempio, $S(7) = 44$:
  - 7 foglie con 2 passi ciascuna
  - 6 nodi interni con 5 passi ciascuno

Soluzione con il Master theorem

- Possiamo ottenere l'ordine di crescita di $S(n)$ direttamente dal Master theorem, senza calcolare $S(n)$ esattamente
- Ricordiamo il **caso 1** del Master theorem:

Sia $S(n) = a\,S(n/b) + f(n)$, con $a \geq 1$ e $b > 1$ costanti e $f(n)$ asintoticamente positiva.

Se $f(n) = O\left(n^{\log_b a - \varepsilon}\right)$ per una qualche costante $\varepsilon > 0$, allora

$$S(n) = \Theta\left(n^{\log_b a}\right).$$

Il caso 1 per `RecursiveSum`

- $S(n) = 2\,S(n/2) + c \quad \rightarrow \quad a = 2,\ b = 2,\ f(n) = c$
- $n^{\log_b a} = n^{\log_2 2} = n^1 = n$
- Confronto: $f(n) = c$ cresce polinomialmente più lentamente di $n \quad \rightarrow \quad$ caso 1
- Ipotesi: $f(n) = c = O\left(n^{1-\varepsilon}\right)$ con $\varepsilon = 1$, poiché $c = c \cdot n^0$
- Conclusione: $S(n) = \Theta\left(n^{\log_2 2}\right) = \Theta(n)$

Come previsto, il risultato è in accordo con la soluzione esatta $S(n) = 7n - 5$.

Tempo di esecuzione nel caso peggiore

- Come nel capitolo «Tempo di esecuzione», facciamo l'ipotesi che ogni elemento dell'array sia memorizzato usando un numero costante fissato di bit
- Allora $|I| = \Theta(n)$, cioè esistono costanti $c_1, c_2 > 0$ tali che $c_1 n \leq |I| \leq c_2 n$ per ogni n sufficientemente grande, e

$$\frac{1}{c_2}|I| \leq n \leq \frac{1}{c_1}|I|.$$

- Poiché $S(n) = 7n - 5 = \Theta(n)$, esistono costanti $a_1, a_2 > 0$ tali che $a_1 n \leq S(n) \leq a_2 n$ per ogni n sufficientemente grande, e quindi

$$\frac{a_1}{c_2}|I| \leq S(n) \leq \frac{a_2}{c_1}|I|.$$

Il tempo di esecuzione dei due algoritmi

Se tutti i valori in ingresso usano un numero costante fissato di bit,
allora il tempo di esecuzione nel caso peggiore dell'algoritmo `RecursiveSum`
(e anche dell'algoritmo `SumIterative`) è

$$T(|I|) = \Theta(|I|).$$

Quindi, entrambi gli algoritmi sono lineari nella dimensione dell'input.

- La ricorsione non rende l'algoritmo più veloce: entrambi gli algoritmi leggono ogni elemento una volta ed eseguono $n-1$ (ricorsivo) oppure n (iterativo) addizioni
- La versione ricorsiva esegue anzi più passi ($7n-5$ contro $3n+4$), a causa del sovraccarico delle chiamate

Memoria aggiuntiva: la pila delle chiamate

- Il tempo non è l'unica risorsa
- Ogni chiamata attiva di una funzione ricorsiva richiede un proprio **record di attivazione** nella **pila delle chiamate** (*call stack*), che contiene i suoi parametri, le sue variabili locali e l'indirizzo di ritorno
- In `RecursiveSum` i parametri e le variabili locali sono l, r, m, s_1, s_2: ogni record di attivazione ha dimensione costante
- Una chiamata è attiva dal momento in cui inizia fino a quando termina. Poiché le chiamate vengono eseguite in profondità, le chiamate attive in ogni istante formano un **cammino** dalla radice dell'albero delle chiamate al nodo corrente

Profondità della ricorsione

Il massimo numero di chiamate contemporaneamente attive
(la **profondità della ricorsione**)
è uguale all'altezza dell'albero delle chiamate più uno.

La memoria aggiuntiva usata da un algoritmo ricorsivo
è (almeno) proporzionale alla profondità della ricorsione.

Osservazione 3: profondità della ricorsione di RecursiveSum

Per ogni $n \geq 1$, l'albero delle chiamate di `RecursiveSum` *su un sotto-array di n elementi ha altezza $\lceil \log_2 n \rceil$. Quindi la profondità della ricorsione è $\lceil \log_2 n \rceil + 1$ e la memoria aggiuntiva usata da* `RecursiveSum` *è $\Theta(\log_2 n)$.*

- La dimostrazione è ancora per induzione forte, sull'altezza $h(n)$ dell'albero delle chiamate

Dimostrazione (1/2)

Sia $h(n)$ l'altezza dell'albero delle chiamate. Allora $h(1) = 0$ e, per $n \geq 2$,

$$h(n) = 1 + \max\left\{h(\lceil n/2 \rceil),\, h(\lfloor n/2 \rfloor)\right\}.$$

Dimostriamo che $h(n) = \lceil \log_2 n \rceil$ per induzione forte.

Per $n = 1$, $\lceil \log_2 1 \rceil = 0$.

Sia $n \geq 2$. Poiché $\lceil \log_2 x \rceil$ è non decrescente in x, per l'ipotesi induttiva il massimo è raggiunto dalla metà più grande, e

$$h(n) = 1 + \left\lceil \log_2 \lceil n/2 \rceil \right\rceil.$$

Dimostrazione (2/2)

Sia $k = \lceil \log_2 n \rceil \geq 1$, cioè

$$2^{k-1} < n \leq 2^k.$$

Dividendo per 2, otteniamo $2^{k-2} < n/2 \leq 2^{k-1}$; poiché 2^{k-1} è un intero, anche

$$2^{k-2} < \lceil n/2 \rceil \leq 2^{k-1},$$

cioè $\lceil \log_2 \lceil n/2 \rceil \rceil = k - 1$. Quindi

$$h(n) = 1 + (k-1) = k = \lceil \log_2 n \rceil.$$

Ogni record di attivazione ha dimensione costante, quindi la memoria aggiuntiva è $\Theta(\lceil \log_2 n \rceil + 1) = \Theta(\log_2 n)$. □

Tempo e memoria dei due algoritmi

- Nell'esempio, $n = 7$ e la profondità della ricorsione è $\lceil \log_2 7 \rceil + 1 = 3 + 1 = 4$
- Per esempio, quando viene eseguita $\texttt{RS}(1, 1)$, le chiamate attive sono $\texttt{RS}(1, 7)$, $\texttt{RS}(1, 4)$, $\texttt{RS}(1, 2)$ e $\texttt{RS}(1, 1)$

algoritmo	numero di passi	memoria aggiuntiva
`SumIterative`	$3n + 4 = \Theta(n)$	$\Theta(1)$
`RecursiveSum`	$7n - 5 = \Theta(n)$	$\Theta(\log_2 n)$

Una ricorsione diversa: decremento di uno

- Dimezzare non è l'unico modo di sommare un array ricorsivamente
- L'array può anche essere suddiviso in modo **sbilanciato**: la somma di $a[1..n]$ è la somma di $a[1..n-1]$ più $a[n]$
- Questa è una ricorsione per *decremento di uno*, il cui pseudocodice è nella prossima slide

Algorithm: RecursiveSumLinear

Input : Un array $a[1..n]$ di $n \geq 1$ numeri interi
Output : La somma $a[1] + a[2] + \cdots + a[n]$

```
1 function RecursiveSumLinear(a, n)
2     if n = 1 then
3         return a[1]                                  // Caso base
4     end
5     s := RecursiveSumLinear(a, n − 1)
6     return s + a[n]
7 end
```

Esegui in Colab: Python · C++ · Java

Correttezza ed equazione di ricorrenza

- La correttezza segue per induzione (semplice) su n:
  - per $n = 1$ l'algoritmo restituisce $a[1]$
  - per $n \geq 2$, per l'ipotesi induttiva $s = a[1] + \cdots + a[n-1]$, e l'algoritmo restituisce $s + a[n] = a[1] + \cdots + a[n]$

- Se $n = 1$, le righe 2 e 3 vengono eseguite una volta: $S(1) = 2$

- Se $n \geq 2$, le righe 2, 5 e 6 vengono eseguite una volta, più i $S(n-1)$ passi della chiamata ricorsiva:

$$S(n) = \begin{cases} 2 & \text{se } n = 1, \\ S(n-1) + 3 & \text{se } n \geq 2. \end{cases}$$

Soluzione per iterazione

- Il Master theorem **non** si può applicare, perché la dimensione del sottoproblema è $n-1$ e non n/b per una costante $b > 1$
- Risolviamo l'equazione per iterazione (srotolamento):

$$\begin{aligned} S(n) &= S(n-1) + 3 = S(n-2) + 2 \cdot 3 = \cdots \\ &= S(1) + (n-1) \cdot 3 = 3n - 1 = \Theta(n). \end{aligned}$$

- L'albero delle chiamate è un **cammino** di n nodi: `RecursiveSumLinear`(a, n) chiama `RecursiveSumLinear`$(a, n-1)$, che chiama `RecursiveSumLinear`$(a, n-2)$, e così via, fino a $n = 1$
- Quando si raggiunge il caso base, tutte le n chiamate sono attive

Il prezzo della ricorsione per decremento di uno

- `RecursiveSumLinear` esegue $S(n) = 3n - 1 = \Theta(n)$ passi, ma la sua profondità di ricorsione è n
- Richiede $\Theta(n)$ di memoria aggiuntiva per la pila delle chiamate, contro $\Theta(\log_2 n)$ di `RecursiveSum` e $\Theta(1)$ di `SumIterative`
- In pratica, per n grande (ad esempio $n = 10^7$) la versione per decremento di uno può esaurire la pila delle chiamate (*stack overflow*), mentre `RecursiveSum` richiede solo

$$\lceil \log_2 10^7 \rceil + 1 = 25$$

chiamate attive

Riepilogo

Punti chiave sulla somma di un array:

- `RecursiveSum` è corretto (induzione forte), effettua $2n - 1$ chiamate ed esegue $7n - 5 = \Theta(n)$ passi: la ricorsione non riduce il lavoro, che è lineare perché ogni elemento deve essere letto

Equazione di ricorrenza:

- $S(n) = 2S(n/2) + c$ ricade nel caso 1 del Master theorem: il costo è dominato dalle n foglie

Memoria:

- La pila delle chiamate costa memoria: $\Theta(\log_2 n)$ per la ricorsione per dimezzamento, $\Theta(n)$ per la ricorsione per decremento di uno, $\Theta(1)$ per il ciclo

Merge Sort

Introduzione: Merge Sort

- Per la somma di un array, il divide et impera non ha dato alcun guadagno in termini di tempo. Incontriamo ora un problema in cui fa la differenza: l'**ordinamento**
- Il Merge Sort suddivide l'array in due metà, le ordina ricorsivamente e **fonde** le due metà ordinate
- Il suo numero di passi è $\Theta(n \log_2 n)$ nel caso peggiore, contro $\Theta(n^2)$ dell'Insertion Sort: il prezzo è $\Theta(n)$ di memoria aggiuntiva

Il problema dell'ordinamento

Dato un array $a[1..n]$ di n numeri naturali,
consideriamo il problema di riordinare gli elementi dell'array
in ordine non decrescente in modo che:

$$a[1] \leq a[2] \leq \cdots \leq a[n].$$

- Il problema è stato introdotto nel capitolo «Algoritmi di ordinamento» (Insertion Sort, Exchange Sort e Selection Sort)
- Nel capitolo «Tempo di esecuzione» abbiamo dimostrato che il numero di passi di `InsertionSort` nel caso peggiore è $S(n) = \frac{3n^2+9n-8}{2} = \Theta(n^2)$
- **Merge Sort**: un algoritmo di ordinamento ricorsivo basato sui confronti e sul paradigma **divide et impera**, con $\Theta(n \log_2 n)$ passi nel caso peggiore

Esempio

Consideriamo il seguente array di $n = 7$ numeri naturali:

$a[i]$	$i = 1$	$i = 2$	$i = 3$	$i = 4$	$i = 5$	$i = 6$	$i = 7$
	9	12	3	16	19	11	12

L'array ordinato è:

$a[i]$	$i = 1$	$i = 2$	$i = 3$	$i = 4$	$i = 5$	$i = 6$	$i = 7$
	3	9	11	12	12	16	19

Divide et impera

- Il Merge Sort ordina un sotto-array $a[l..r]$ come segue:
  1. **Divide**: si suddivide $a[l..r]$ nelle due metà $a[l..m]$ e $a[m+1..r]$, dove $m = \lfloor (l+r)/2 \rfloor$;
  2. **Impera**: si ordinano ricorsivamente le due metà;
  3. **Combina**: si **fondono** (*merge*) le due metà ordinate in un unico sotto-array ordinato $a[l..r]$.

- Il **caso base** è un sotto-array con un solo elemento ($l = r$), che è già ordinato: non occorre fare nulla

- Tutto il lavoro è svolto nel passo **combina**, dalla procedura `Merge`

- Osservazione chiave: due sequenze *ordinate* si possono fondere in modo molto efficiente. Il più piccolo elemento del risultato è il minore tra i due primi elementi, e così via

Questo è lo pseudocodice dell'algoritmo `MergeSort`.
Per ordinare l'intero array $a[1..n]$, chiamiamo `MergeSort`$(a, 1, n)$.

Algorithm: MergeSort

Input : Un array a e due indici $l \leq r$
Output : Il sotto-array $a[l..r]$ ordinato in ordine non decrescente

```
1 function MergeSort(a, l, r)
2     if l < r then
3         m := ⌊(l + r)/2⌋                          // Divide
4         MergeSort(a, l, m)      // Impera (metà sinistra)
5         MergeSort(a, m + 1, r)  // Impera (metà destra)
6         Merge(a, l, m, r)                         // Combina
7     end
8 end
```

Esegui in Colab: Python · C++ · Java

La procedura `Merge`

- Se $k = r - l + 1 \geq 2$, le due metà $a[l..m]$ e $a[m+1..r]$ hanno $\lceil k/2 \rceil$ e $\lfloor k/2 \rfloor$ elementi, entrambi compresi tra 1 e $k-1$: la ricorsione raggiunge sempre il caso base
- La procedura `Merge`(a, l, m, r) suppone che $a[l..m]$ e $a[m+1..r]$ siano ordinati
- Li copia in due array ausiliari L e R, e poi sposta ripetutamente in a il minore tra i primi due elementi non ancora copiati
- Il suo pseudocodice è lungo: lo spezziamo su quattro slide, con la numerazione delle righe che continua

Algorithm: Merge (1/4)

Input : Un array a e indici $l \leq m < r$ tali che $a[l..m]$ e $a[m+1..r]$ siano ordinati

Output : Il sotto-array $a[l..r]$ ordinato in ordine non decrescente

```
1  function Merge(a, l, m, r)
2      n1 := m − l + 1
3      n2 := r − m
4      L := nuovo array[1..n1], R := nuovo array[1..n2]
5      for i := 1 to n1 do
6          L[i] := a[l + i − 1]
7      end
8      for j := 1 to n2 do
9          R[j] := a[m + j]
10     end
11     i := 1
12     j := 1
```

Algorithm: Merge (2/4)

function *Merge(a, l, m, r)* (continua)

```
13    k := l
14    while i ≤ n1 and j ≤ n2 do
15        if L[i] ≤ R[j] then
16            a[k] ← L[i]
17            i ← i + 1
18        else
19            a[k] ← R[j]
20            j ← j + 1
21        end
22        k ← k + 1
23    end
```

Algorithm: Merge (3/4)

function *Merge(a, l, m, r)* (continua)

```
24      while i ≤ n_1 do                    // Copia il resto di L
25          a[k] ← L[i]
26          i ← i + 1
27          k ← k + 1
28      end
```

Algorithm: Merge (4/4)

function *Merge(a, l, m, r)* (continua)

```
29      while j ≤ n_2 do                    // Copia il resto di R
30          a[k] ← R[j]
31          j ← j + 1
32          k ← k + 1
33      end
34  end
```

Esegui in Colab: Python · C++ · Java

Come funziona Merge (1/2)

- Le righe 2–9 copiano $a[l..m]$ in $L[1..n_1]$ e $a[m+1..r]$ in $R[1..n_2]$
- Le copie sono necessarie, perché le posizioni $a[l..r]$ vengono sovrascritte durante la fusione
- Gli indici i e j puntano ai primi elementi di L e R non ancora ricopiati, e k punta alla prossima posizione di a da riempire

Come funziona `Merge` (2/2)

- Il primo ciclo while (riga 14) confronta $L[i]$ e $R[j]$ e copia il minore in $a[k]$
- In caso di parità, viene copiato per primo l'elemento di L: questo rende il Merge Sort **stabile**
- Non appena uno dei due array si esaurisce, gli elementi rimanenti dell'altro (che sono già ordinati, e non più piccoli di quelli già copiati) vengono copiati in a dal ciclo while della riga 24 o da quello della riga 29
- Solo uno di questi due cicli esegue almeno un'iterazione

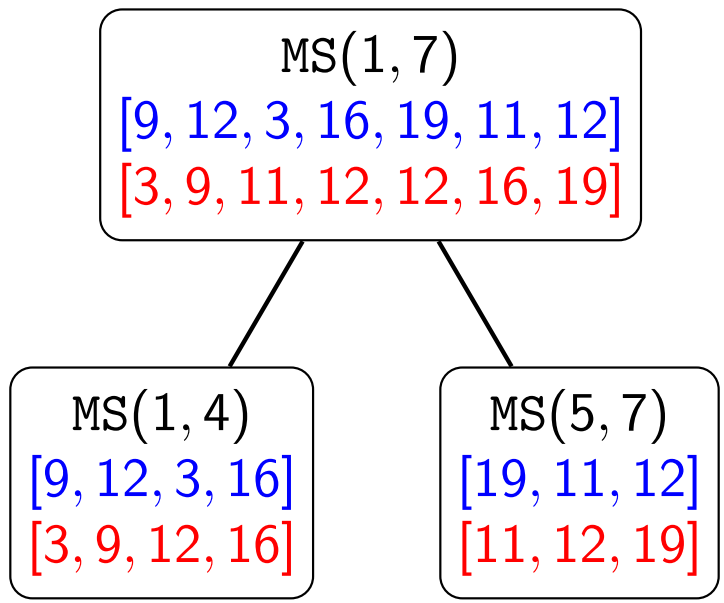
Esempi

Eseguiamo ora il Merge Sort sull'array
$a = [9, 12, 3, 16, 19, 11, 12]$:

guardiamo prima l'intero albero delle chiamate,
poi le due fasi dell'algoritmo,
e infine una singola chiamata di `Merge`.

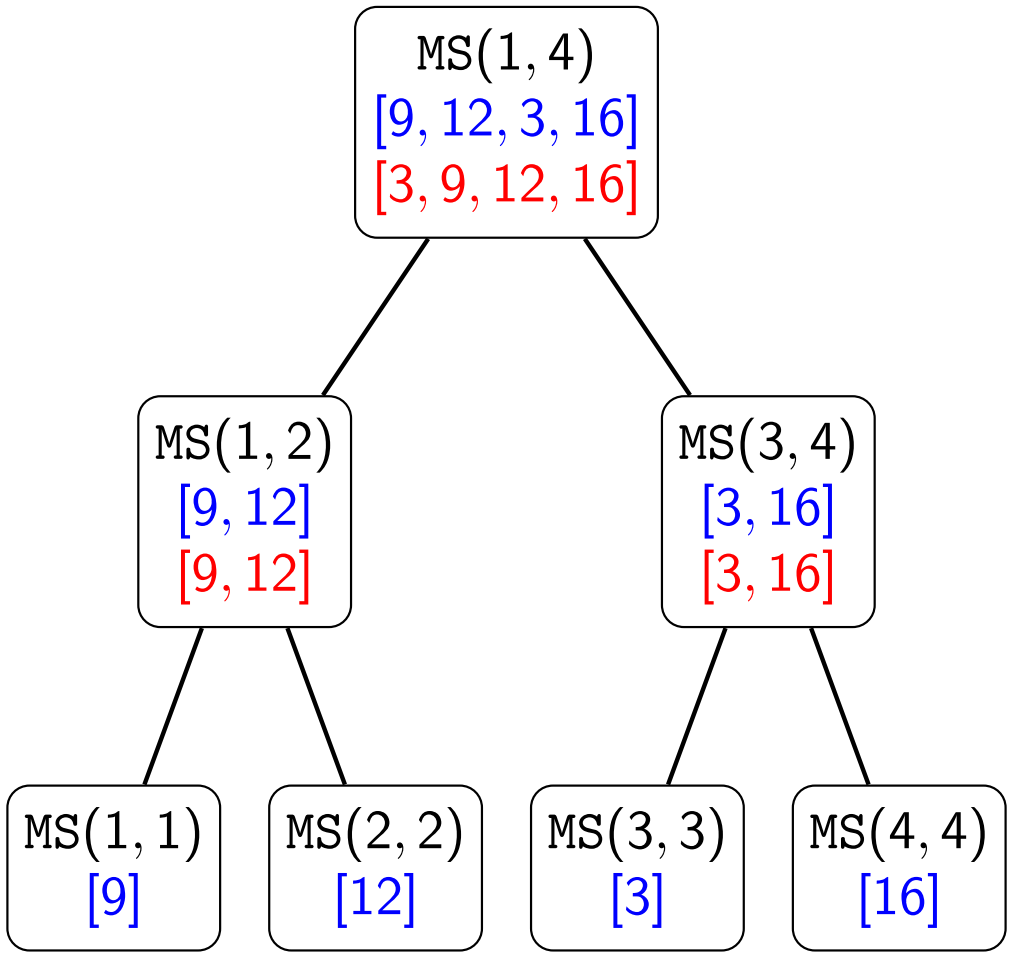
Esempio: albero delle chiamate (1/4)

- L'albero delle chiamate di `MergeSort`$(a, 1, 7)$: ogni nodo `MS`(l, r) è una chiamata sul sotto-array $a[l..r]$
- In ogni nodo, in blu il sotto-array ricevuto dalla chiamata e in rosso il sotto-array ordinato prodotto dalla chiamata (dopo la sua `Merge`)
- Prima la radice e i suoi due figli, poi i sottoalberi di `MS`$(1, 4)$ e di `MS`$(5, 7)$



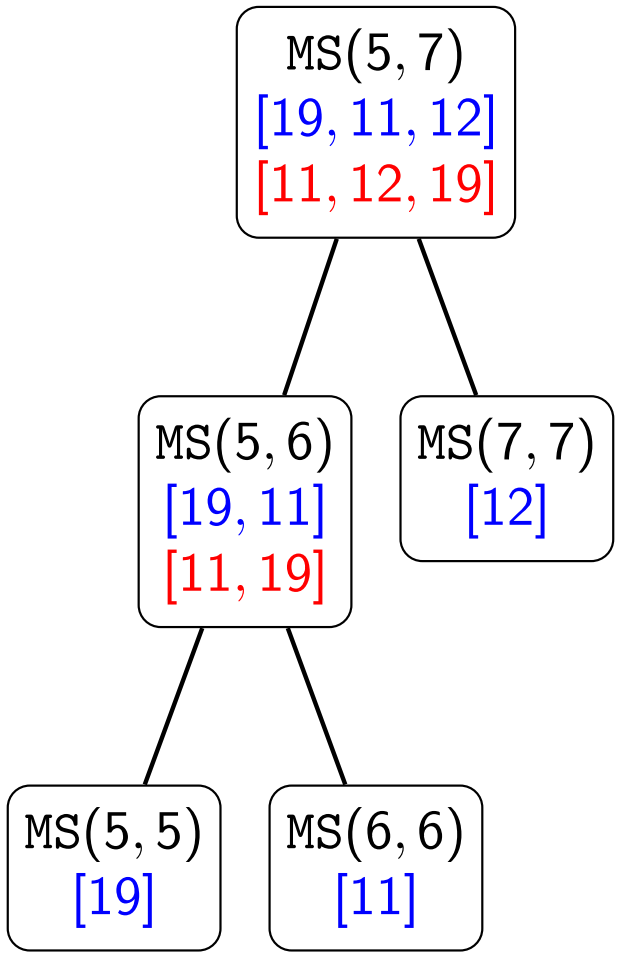
Esempio: albero delle chiamate (2/4)

Il sottoalbero della chiamata $\texttt{MS}(1, 4)$:



Esempio: albero delle chiamate (3/4)

Il sottoalbero della chiamata $\texttt{MS}(5, 7)$:



Esempio: albero delle chiamate (4/4)

Le chiamate di `Merge` vengono eseguite nel seguente ordine (in profondità), e questo è il contenuto dell'array dopo ciascuna di esse (il sotto-array modificato da ciascuna chiamata è in rosso):

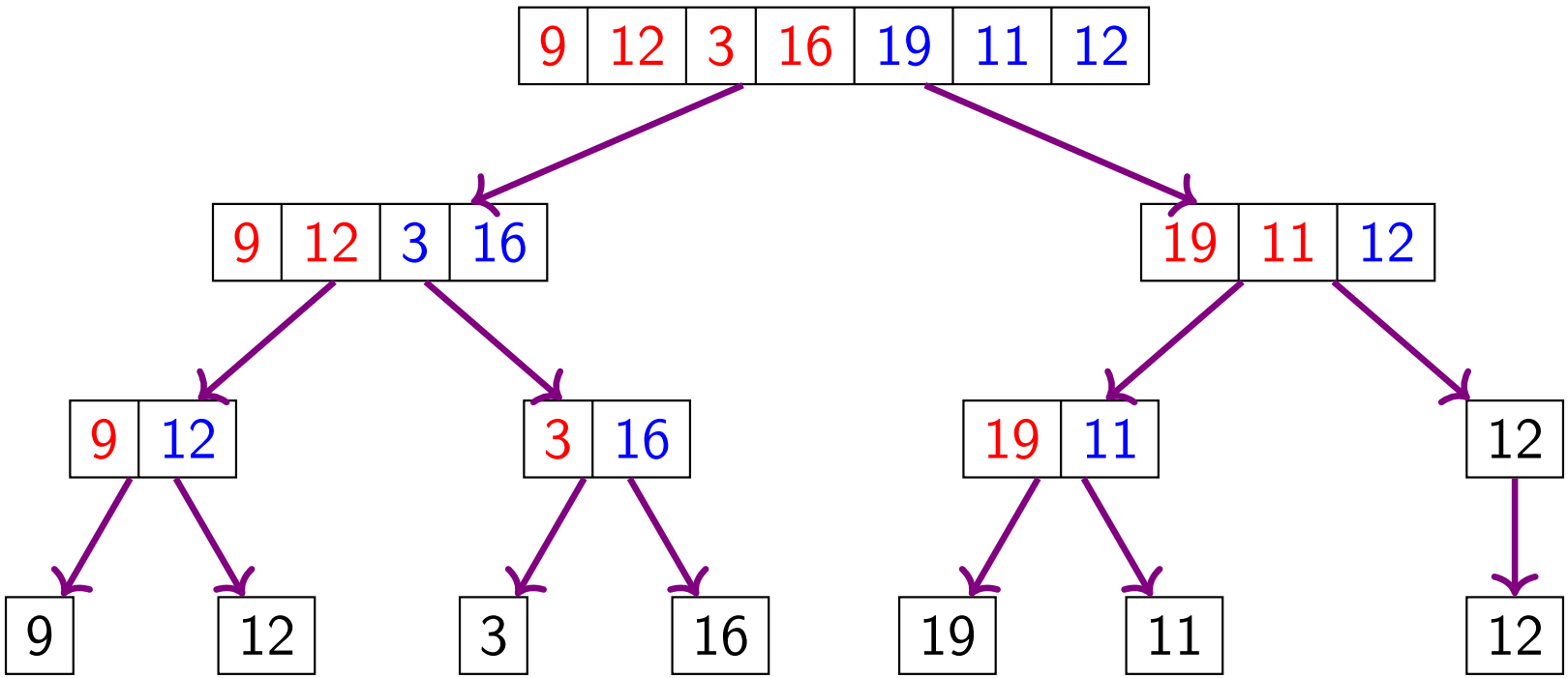
chiamata	sotto-array fusi	array a dopo la chiamata
`Merge`$(a, 1, 1, 2)$	$[9]$ e $[12]$	$[9, 12, 3, 16, 19, 11, 12]$
`Merge`$(a, 3, 3, 4)$	$[3]$ e $[16]$	$[9, 12, 3, 16, 19, 11, 12]$
`Merge`$(a, 1, 2, 4)$	$[9, 12]$ e $[3, 16]$	$[3, 9, 12, 16, 19, 11, 12]$
`Merge`$(a, 5, 5, 6)$	$[19]$ e $[11]$	$[3, 9, 12, 16, 11, 19, 12]$
`Merge`$(a, 5, 6, 7)$	$[11, 19]$ e $[12]$	$[3, 9, 12, 16, 11, 12, 19]$
`Merge`$(a, 1, 4, 7)$	$[3, 9, 12, 16]$ e $[11, 12, 19]$	$[3, 9, 11, 12, 12, 16, 19]$

Esempio: fasi di divisione e di combinazione (1/3)

- Consideriamo di nuovo l'array $a = [9, 12, 3, 16, 19, 11, 12]$
- La fase di **divisione** (frecce viola): ogni sotto-array viene suddiviso nella sua metà sinistra (in rosso) e nella sua metà destra (in blu), fino a ottenere sotto-array di un elemento
- La fase di **combinazione** (frecce arancioni): coppie di sotto-array ordinati vengono fuse in un sotto-array ordinato

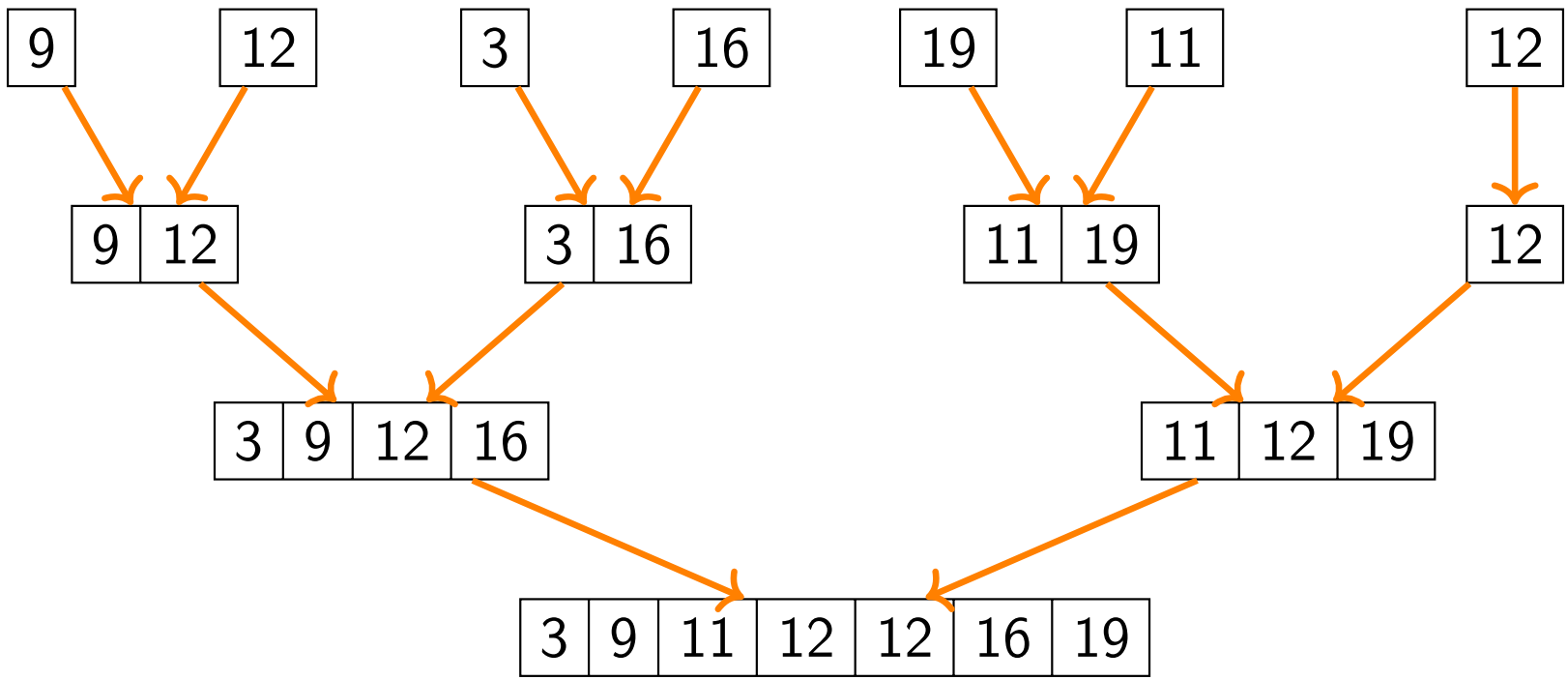
Esempio: fasi di divisione e di combinazione (2/3)

La fase di **divisione**:



Esempio: fasi di divisione e di combinazione (3/3)

La fase di **combinazione**:



Esempio: esecuzione di `Merge` (1/2)

- Consideriamo l'ultima chiamata `Merge`$(a, 1, 4, 7)$ dell'albero delle chiamate
- Prima della chiamata $a = [3, 9, 12, 16, 11, 12, 19]$, con $a[1..4]$ e $a[5..7]$ ordinati
- Allora $n_1 = 4$, $n_2 = 3$ e

$$L = [3, 9, 12, 16], \qquad R = [11, 12, 19].$$

- Nella prossima slide, l'esecuzione dei cicli while (una riga per ogni elemento copiato in a)

Esempio: esecuzione di `Merge` (2/2)

k	i	j	$L[i]$	$R[j]$	confronto	azione
1	1	1	3	11	$3 \leq 11$	$a[1] \leftarrow 3$, $i = 2$
2	2	1	9	11	$9 \leq 11$	$a[2] \leftarrow 9$, $i = 3$
3	3	1	12	11	$12 > 11$	$a[3] \leftarrow 11$, $j = 2$
4	3	2	12	12	$12 \leq 12$	$a[4] \leftarrow 12$ (da L), $i = 4$
5	4	2	16	12	$16 > 12$	$a[5] \leftarrow 12$ (da R), $j = 3$
6	4	3	16	19	$16 \leq 19$	$a[6] \leftarrow 16$, $i = 5$
7	5	3	–	19	L esaurito	$a[7] \leftarrow 19$, $j = 4$

- Dopo 6 iterazioni del primo ciclo while (e 6 confronti), $i = 5 > n_1$: l'array L è esaurito, e l'ultimo elemento di R viene copiato dal ciclo while della riga 29
- Il risultato è $a = [3, 9, 11, 12, 12, 16, 19]$

Correttezza

- La correttezza del Merge Sort poggia su due fatti:
  - `Merge` fonde correttamente due sotto-array ordinati: una proprietà di un ciclo, dimostrata con un'**invariante di ciclo**
  - la ricorsione è corretta: dimostrata per **induzione forte**, come per `RecursiveSum`

- Un'invariante di ciclo è una proprietà che vale all'inizio di ogni iterazione di un ciclo. Come nel capitolo «Correttezza», mostriamo che l'invariante
  - vale prima della prima iterazione (*inizializzazione*)
  - se vale prima di un'iterazione, vale anche prima della successiva (*mantenimento*)

  e che cosa ci fornisce quando il ciclo termina (*terminazione*)

Osservazione 4: invariante di ciclo di Merge

Siano $a[l..m]$ e $a[m+1..r]$ ordinati. All'inizio di ogni iterazione del ciclo while della riga 14 di `Merge`*:*

(I1) *il sotto-array $a[l..k-1]$ contiene i $k-l$ elementi di $L[1..i-1]$ e $R[1..j-1]$, in ordine non decrescente;*

(I2) *ogni elemento di $a[l..k-1]$ è minore o uguale a ogni elemento di $L[i..n_1]$ e di $R[j..n_2]$.*

Alla fine, `Merge` *termina e $a[l..r]$ contiene gli elementi originariamente in $a[l..r]$, ordinati in ordine non decrescente.*

Dimostrazione (1/4)

Dopo le righe 2–9, $L[1..n_1]$ e $R[1..n_2]$ sono copie dei sotto-array ordinati $a[l..m]$ e $a[m+1..r]$, quindi sono ordinati.

Inizializzazione. Prima della prima iterazione $k = l$, $i = 1$, $j = 1$: il sotto-array $a[l..k-1]$ è vuoto, e (I1) e (I2) valgono banalmente.

Dimostrazione (2/4)

Mantenimento. Supponiamo (I1) e (I2) all'inizio di un'iterazione, e sia $L[i] \leq R[j]$ (l'altro caso è simmetrico).

- Poiché L e R sono ordinati,

$$L[i] \leq L[i+1..n_1] \qquad \text{e} \qquad L[i] \leq R[j] \leq R[j+1..n_2]:$$

  $L[i]$ è il più piccolo elemento non ancora copiato

- Per (I2), $L[i]$ è maggiore o uguale a tutti gli elementi di $a[l..k-1]$, quindi dopo $a[k] \leftarrow L[i]$ il sotto-array $a[l..k]$ è ordinato e contiene $L[1..i]$ e $R[1..j-1]$

Dimostrazione (3/4)

- Dopo $i \leftarrow i + 1$ e $k \leftarrow k + 1$, (I1) vale di nuovo
- Vale anche (I2), perché gli elementi non ancora copiati sono un sottoinsieme dei precedenti, e il nuovo elemento $a[k-1] = L[i-1]$ è minore o uguale a tutti loro

Dimostrazione (4/4)

Terminazione. Ogni iterazione aumenta $i + j$ di uno, quindi il ciclo termina dopo al più $n_1 + n_2$ iterazioni, con $i = n_1 + 1$ oppure $j = n_2 + 1$.

Supponiamo $i = n_1 + 1$ (l'altro caso è simmetrico).

- Il ciclo della riga 24 viene saltato, e il ciclo della riga 29 copia $R[j..n_2]$, in questo ordine, in $a[k..r]$
- Poiché $R[j..n_2]$ è ordinato e, per (I2), i suoi elementi non sono più piccoli di quelli di $a[l..k-1]$, l'intero sotto-array $a[l..r]$ è ordinato
- Per (I1), contiene esattamente gli elementi di L e R, cioè gli elementi originariamente in $a[l..r]$

Correttezza di `MergeSort`

Ora che `Merge` è corretta,
la correttezza della ricorsione segue per induzione forte.

Osservazione 5: correttezza di MergeSort

Per ogni coppia di indici $l \leq r$, la chiamata `MergeSort`(a, l, r) *termina e riordina gli elementi di* $a[l..r]$ *in ordine non decrescente (senza modificare il resto dell'array).*

Dimostrazione (1/2)

Usiamo l'induzione forte sul numero $k = r - l + 1 \geq 1$ di elementi del sotto-array.

Caso base ($k = 1$). Allora $l = r$, la condizione della riga 2 è falsa e non si fa nulla: un sotto-array di un elemento è ordinato.

Passo induttivo ($k \geq 2$). Supponiamo che l'enunciato valga per tutti i sotto-array con meno di k elementi. Poiché $l < r$, l'algoritmo calcola $m = \lfloor (l + r)/2 \rfloor$, con $l \leq m < r$.

Dimostrazione (2/2)

- I sotto-array $a[l..m]$ e $a[m+1..r]$ hanno $\lceil k/2 \rceil$ e $\lfloor k/2 \rfloor$ elementi, entrambi compresi tra 1 e $k-1$
- Per l'ipotesi induttiva, dopo le righe 4 e 5 sia $a[l..m]$ sia $a[m+1..r]$ sono ordinati (e contengono gli stessi elementi di prima)
- Quindi l'ipotesi di `Merge` è soddisfatta, e per l'Osservazione 4, dopo la riga 6 il sotto-array $a[l..r]$ è ordinato

□

Numero di passi di `Merge`

- Sia $n = r - l + 1 = n_1 + n_2$ il numero di elementi da fondere
- Sia t il numero di iterazioni del ciclo while della riga 14. Tra queste, t_L copiano un elemento di L e t_R copiano un elemento di R, con $t_L + t_R = t$
- L'intestazione di un ciclo for da 1 a n_1 viene eseguita $n_1 + 1$ volte (l'ultimo test fallisce)
- La riga `else` fa parte dell'`if` della riga 15 e non viene contata separatamente

Numero di esecuzioni di ciascuna riga

Il numero di volte che ciascuna riga di codice viene eseguita (le righe di uno stesso gruppo lo stesso numero di volte):

righe	esecuzioni
2, 3, 4	1
5	$n_1 + 1$
6	n_1
8	$n_2 + 1$
9	n_2
11, 12, 13	1
14	$t + 1$
15	t

righe	esecuzioni
16, 17	t_L
19, 20	t_R
22	t
24	$(n_1 - t_L) + 1$
25, 26, 27	$n_1 - t_L$
29	$(n_2 - t_R) + 1$
30, 31, 32	$n_2 - t_R$

Numero totale di passi di `Merge`

Quindi, il numero totale di passi di `Merge` è

$$\begin{aligned} &3 + (2n_1 + 1) + (2n_2 + 1) + 3 + (t + 1) + t + 2t_L + 2t_R + t \\ &\quad + 4(n_1 - t_L) + 1 + 4(n_2 - t_R) + 1 \\ &= 11 + 2n + 5t + 4(n - t) = 6n + t + 11. \end{aligned}$$

- Ogni iterazione del primo ciclo while copia un elemento in a, quindi $t \leq n$. Più precisamente:
  - il ciclo si ferma non appena uno dei due array si esaurisce, quindi $t \geq \min\{n_1, n_2\}$
  - quando il ciclo si ferma, l'altro array contiene ancora almeno un elemento, quindi $t \leq n - 1$

Limiti al numero di passi di Merge

- Quindi il numero di passi di Merge su n elementi è compreso tra $6n + 11 + \min\{n_1, n_2\}$ e $7n + 10$; in particolare, è compreso tra $6n + 11$ e $7n + 10$
- Il caso peggiore $t = n - 1$ si verifica, per esempio, nell'esecuzione di Merge vista sopra ($n = 7$, $t = 6$, $6 \cdot 7 + 6 + 11 = 59$ passi)

Il numero di passi di Merge su $n = r - l + 1$ elementi è $6n + t + 11$ e soddisfa

$$6n+11 \leq S_{\texttt{Merge}}(n) \leq 7n+10, \qquad \text{quindi} \qquad S_{\texttt{Merge}}(n) = \Theta(n).$$

Il numero di confronti tra elementi (riga 15) è t, con $\min\{n_1, n_2\} \leq t \leq n - 1$.

L'equazione di ricorrenza di `MergeSort` (1/2)

- Sia $S(n)$ il numero di passi eseguiti da `MergeSort` nel caso peggiore su un sotto-array di $n = r - l + 1$ elementi
- Come per la somma di un array, una riga che contiene una chiamata conta 1 passo più i passi della funzione chiamata
- Se $n = 1$, viene eseguita solo la riga 2: $S(1) = 1$
- Se $n \geq 2$:
  - La riga 2 viene eseguita 1 volta
  - La riga 3 viene eseguita 1 volta
  - La riga 4 viene eseguita 1 volta, più i $S(\lceil n/2 \rceil)$ passi della chiamata
  - La riga 5 viene eseguita 1 volta, più i $S(\lfloor n/2 \rfloor)$ passi della chiamata
  - La riga 6 viene eseguita 1 volta, più i $S_{\texttt{Merge}}(n)$ passi di `Merge`

L'equazione di ricorrenza di `MergeSort` (2/2)

- Quindi

$$S(n) = S(\lceil n/2 \rceil) + S(\lfloor n/2 \rfloor) + f(n), \qquad f(n) = 5 + S_{\texttt{Merge}}(n).$$

- Dall'analisi di `Merge`, $6n + 16 \leq f(n) \leq 7n + 15$
- Poiché $7n + 15 \leq 15n$ per $n \geq 2$, abbiamo

$$6n \leq f(n) \leq 15\,n \qquad \text{per ogni } n \geq 2,$$

cioè $f(n) = \Theta(n)$

L'equazione di ricorrenza di `MergeSort`

Il numero di passi di `MergeSort` è descritto dall'equazione di ricorrenza

$$S(n) = \begin{cases} 1 & \text{se } n = 1, \\ S(\lceil n/2 \rceil) + S(\lfloor n/2 \rfloor) + \Theta(n) & \text{se } n \geq 2, \end{cases}$$

che, ignorando le parti intere inferiori e superiori, scriviamo come

$$S(n) = 2\,S(n/2) + \Theta(n).$$

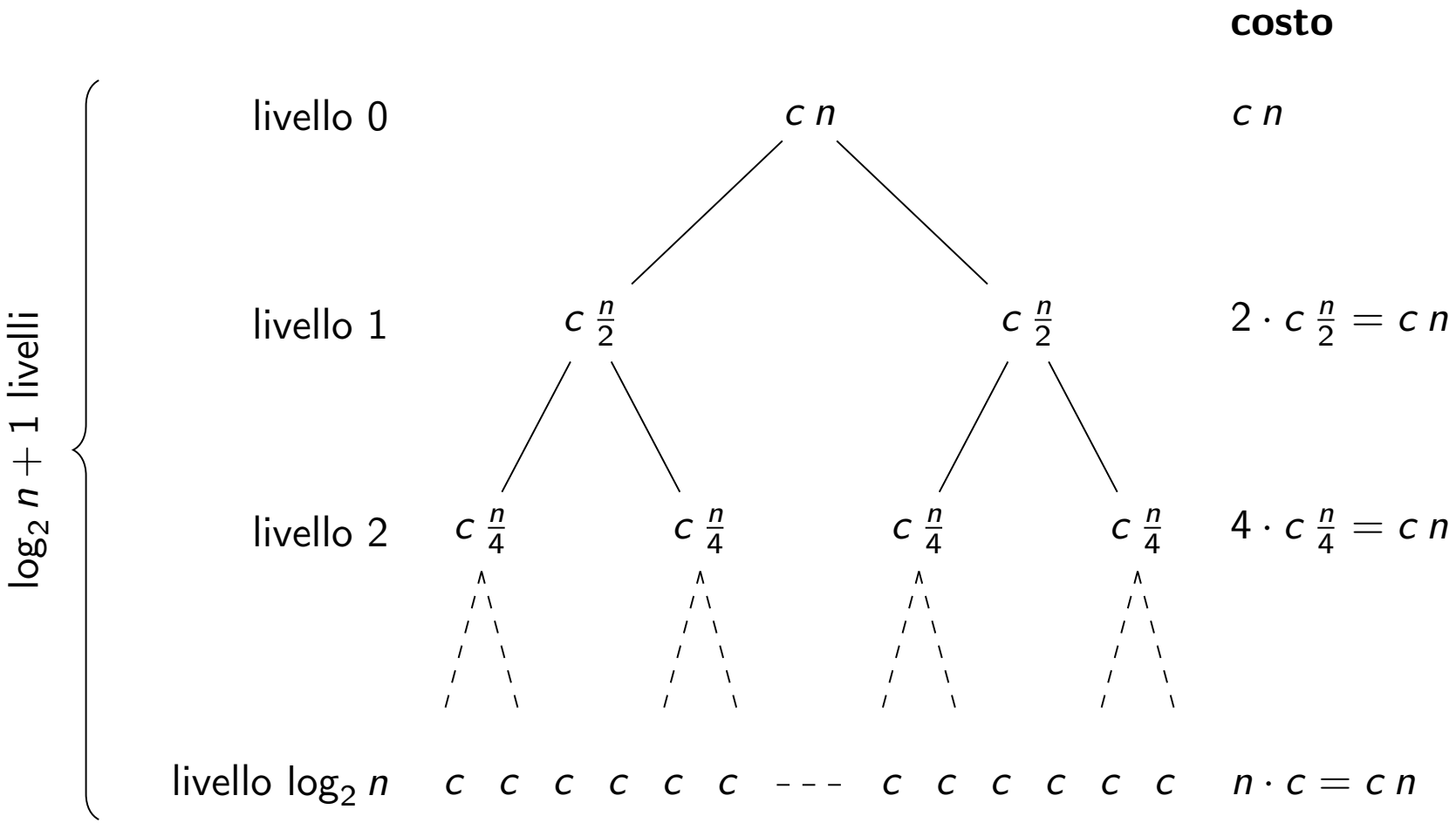
Soluzione con l'albero di ricorsione

- Supponiamo che $n = 2^h$ sia una potenza esatta di 2, e sia $c = 15$ la costante tale che $f(n) \leq c\,n$ (con $S(1) = 1 \leq c$)
- Limitiamo dapprima $S(n)$ dall'alto con l'albero di ricorsione di

$$S(n) \leq 2\,S(n/2) + c\,n,$$

  dove ogni nodo contiene il costo di una chiamata, escluso il costo delle sue chiamate ricorsive
- L'albero è nella prossima slide

L'albero di ricorsione di $S(n) \leq 2\,S(n/2) + c\,n$



Il costo dei livelli

- Il livello superiore ha costo $c\,n$; il livello successivo ha costo $c\frac{n}{2} + c\frac{n}{2} = c\,n$; quello dopo ha costo $4 \cdot c\frac{n}{4} = c\,n$, e così via
- In generale, il livello i ha 2^i nodi, ciascuno corrispondente a un sotto-array di $n/2^i$ elementi e con un contributo di costo al più $c\,n/2^i$, quindi ogni livello costa al più

$$2^i \cdot c\,\frac{n}{2^i} = c\,n.$$

- Il livello più basso $\log_2 n$ ha n foglie (sotto-array di un elemento), ciascuna con costo $S(1) = 1 \leq c$: il suo costo è al più $c\,n$
- I livelli sono $0, 1, \ldots, \log_2 n$, cioè $\log_2 n + 1$. Sommando i costi di tutti i livelli:

$$S(n) \leq c\,n\,(\log_2 n + 1) = c\,n \log_2 n + c\,n.$$

$S(n) = \Theta(n \log_2 n)$

- Allo stesso modo, usando $f(n) \geq 6n$, ogni livello interno costa almeno $6n$, e ci sono $\log_2 n$ livelli interni: $S(n) \geq 6\, n \log_2 n$
- Quindi, per n potenza di 2,

$$6\, n \log_2 n \;\leq\; S(n) \;\leq\; 15\, n \log_2 n + 15\, n,$$

cioè $S(n) = \Theta(n \log_2 n)$, poiché

$$\lim_{n\to\infty} \frac{15\, n \log_2 n + 15\, n}{n \log_2 n} = \lim_{n\to\infty} \left(15 + \frac{15}{\log_2 n}\right) = 15 \in \mathbb{R}_{>0}.$$

Tutti i livelli hanno lo stesso costo

- A differenza dell'albero di ricorsione della somma di un array, in cui il costo dei livelli raddoppia a ogni livello, qui **tutti i livelli hanno lo stesso costo** $c\,n$: il totale è il costo di un livello per il numero di livelli
- Questa è esattamente la situazione del **caso 2** del Master theorem

Soluzione con il Master theorem

Ricordiamo il **caso 2** del Master theorem:

Sia $S(n) = a\,S(n/b) + f(n)$, con $a \geq 1$ e $b > 1$ costanti
e $f(n)$ asintoticamente positiva.

Se $f(n) = \Theta\left(n^{\log_b a} \log^k n\right)$ per una qualche costante $k \geq 0$, allora

$$S(n) = \Theta\left(n^{\log_b a} \log^{k+1} n\right).$$

Il caso 2 per MergeSort

- $S(n) = 2\,S(n/2) + \Theta(n) \quad \rightarrow \quad a = 2,\ b = 2,\ f(n) = \Theta(n)$
- $n^{\log_b a} = n^{\log_2 2} = n$
- Confronto: $f(n)$ e $n^{\log_b a}$ crescono allo stesso modo $\rightarrow$ caso 2
- Ipotesi: $f(n) = \Theta(n) = \Theta\left(n^{\log_2 2} \log^0 n\right)$ con $k = 0$
- Conclusione: $S(n) = \Theta\left(n^{\log_2 2} \log^1 n\right) = \Theta(n \log n)$

- La base del logaritmo è irrilevante all'interno di Θ, perché $\log_b n = \frac{\log_2 n}{\log_2 b}$ differisce da $\log_2 n$ per un fattore costante
- Quindi $S(n) = \Theta(n \log_2 n)$, in accordo con l'albero di ricorsione

Tempo di esecuzione nel caso peggiore (1/2)

- Facciamo l'ipotesi che ogni elemento dell'array sia memorizzato usando un numero costante fissato di bit
- Allora $|I| = \Theta(n)$: esistono costanti $c_1, c_2 > 0$ tali che, per ogni n sufficientemente grande,

$$\frac{1}{c_2}|I| \leq n \leq \frac{1}{c_1}|I|.$$

- Passando ai logaritmi,
  $\log_2 |I| - \log_2 c_2 \leq \log_2 n \leq \log_2 |I| - \log_2 c_1$, quindi, per $|I|$ sufficientemente grande,

$$\frac{1}{2}\log_2 |I| \leq \log_2 n \leq 2\log_2 |I|.$$

Tempo di esecuzione nel caso peggiore (2/2)

- Moltiplicando le due catene di disuguaglianze (tutti i termini sono positivi),

$$\frac{1}{2c_2}\,|I|\log_2|I| \;\leq\; n\log_2 n \;\leq\; \frac{2}{c_1}\,|I|\log_2|I|,$$

  cioè $n\log_2 n = \Theta(|I|\log_2|I|)$

- Poiché $S(n) = \Theta(n\log_2 n)$, otteniamo $S(n) = \Theta(|I|\log_2|I|)$

Se tutti i valori in ingresso usano un numero costante fissato di bit,
allora il tempo di esecuzione nel caso peggiore dell'algoritmo `MergeSort` è

$$T(|I|) = \Theta(|I|\log|I|).$$

Memoria aggiuntiva

- `Merge`(a, l, m, r) alloca gli array ausiliari L e R, con $n_1 + n_2 = r - l + 1$ elementi in totale, che vengono rilasciati quando `Merge` termina
- `Merge` viene chiamata solo dopo che entrambe le chiamate ricorsive sono terminate, quindi in ogni istante è attiva al più una `Merge`, e gli array ausiliari più grandi sono quelli della prima chiamata `MergeSort`$(a, 1, n)$, con n elementi in totale
- Inoltre, la pila delle chiamate contiene al più $\lceil \log_2 n \rceil + 1$ record di attivazione di dimensione costante (l'albero delle chiamate ha la stessa forma di quello di `RecursiveSum`, Osservazione 3): memoria $\Theta(\log_2 n)$

Il Merge Sort non ordina sul posto

`MergeSort` usa

$$\Theta(n) + \Theta(\log_2 n) = \Theta(n)$$

di memoria aggiuntiva: **non** è un algoritmo di ordinamento *in place*.

Al contrario, `InsertionSort` ordina sul posto,
con $\Theta(1)$ di memoria aggiuntiva.

Confronto con Insertion Sort

- Nel caso peggiore, `InsertionSort` esegue $\Theta(n^2)$ passi, mentre `MergeSort` esegue $\Theta(n \log_2 n)$ passi. Poiché

$$\lim_{n\to\infty} \frac{n \log_2 n}{n^2} = \lim_{n\to\infty} \frac{\log_2 n}{n} = 0,$$

  il Merge Sort è asintoticamente molto più veloce

- Per farsi un'idea della differenza, ignoriamo le costanti moltiplicative e supponiamo che un calcolatore esegua 10^9 passi al secondo: si veda la prossima slide

n^2 contro $n \log_2 n$

n	n^2	$n \log_2 n$	tempo per n^2 passi	tempo per $n \log_2 n$ passi
10	100	≈ 33	10^{-7} s	$\approx 3.3 \cdot 10^{-8}$ s
10^3	10^6	$\approx 9\,966$	10^{-3} s	$\approx 10^{-5}$ s
10^6	10^{12}	$\approx 1.99 \cdot 10^7$	10^3 s ≈ 16.7 min	≈ 0.02 s

- Per array piccoli la differenza è trascurabile (e, a causa delle costanti e del costo delle chiamate ricorsive, `InsertionSort` può anche essere più veloce)
- Per array grandi, il Merge Sort è enormemente più veloce: con $n = 10^6$ elementi è circa 50 000 volte più veloce

Riepilogo

Punti chiave sul Merge Sort:

- Tutto il lavoro è svolto da `Merge`, che fonde due sotto-array ordinati di dimensione totale n in $6n + t + 11 = \Theta(n)$ passi
- $S(n) = 2S(n/2) + \Theta(n)$ ricade nel caso 2 del Master theorem: $\log_2 n + 1$ livelli di costo $\Theta(n)$ ciascuno, $S(n) = \Theta(n \log_2 n)$
- Il prezzo è la memoria: $\Theta(n)$ per gli array ausiliari

algoritmo	passi nel caso peggiore	memoria aggiuntiva
`InsertionSort`	$\Theta(n^2)$	$\Theta(1)$
`MergeSort`	$\Theta(n \log_2 n)$	$\Theta(n)$

Riepilogo della lezione (1/3)

Lo schema dell'analisi:

- pseudocodice e albero delle chiamate
- correttezza per induzione forte sulla dimensione del sottoproblema
- equazione di ricorrenza, risolta con l'albero di ricorsione e con il Master theorem
- tempo di esecuzione in funzione di $|I|$
- memoria aggiuntiva, compresa la pila delle chiamate

Riepilogo della lezione (2/3)

La somma di un array:

- `RecursiveSum`: $S(n) = 2S(n/2) + c$, $7n - 5 = \Theta(n)$ passi, memoria $\Theta(\log_2 n)$
- `RecursiveSumLinear`: $S(n) = S(n-1) + 3$, $3n - 1 = \Theta(n)$ passi, memoria $\Theta(n)$
- la ricorsione non dà alcun guadagno in tempo rispetto a `SumIterative`, e costa memoria

Riepilogo della lezione (3/3)

Merge Sort:

- si divide in due metà, le si ordina ricorsivamente, le si **fonde** in $\Theta(n)$ passi
- correttezza: un'invariante di ciclo per `Merge`, l'induzione forte per la ricorsione
- $S(n) = 2S(n/2) + \Theta(n) = \Theta(n \log_2 n)$, contro $\Theta(n^2)$ di `InsertionSort`
- $\Theta(n)$ di memoria aggiuntiva: non ordina sul posto

Prossima lezione:

- la ricerca binaria ricorsiva (una chiamata ricorsiva) e il problema del sotto-array di somma massima